Exercise 02: Intro CocoTB Exercises

These exercises will introduce you to:

- The (Make) flow for running CocoTB simulations
- CocoTB constructs

0.1 Python environment setup

Before starting the exercise make sure to create the virtual environment and activate it.

```
# Create the virtual env
<username>@<servername> ~/02207-repo$ source bin/venv.src.me
# Activate the virtual env
<username>@<servername> ~/02207-repo$ source .venv/bin/activate
```

0.1.1 Exercise A: Simple *cocotb* Test for Adder Design

Objective: *Introduction to the cocotb tests.*

Task: *Run cocotb tests for simple RTL and view waveforms.*

Inspect the directories and files in the adder-test example in

`lecture02/exercises/E02_intro_cocotb_exercisesE02_intro_cocotb_exercises/A_example_adder`

- What do the individual variables in the Makefile control? Read the docs at `https://docs.cocotb.org/en/v2.0.1/building.html`
- Why does the Makefile extend the `PYTHONPATH`? Hint: `PYTHONPATH` is a Python-native environment variable, not part of the cocotb build system.

To run the example:

1. Activate the virtual environment.
2. Navigate to the `test`-folder in a shell.
3. Run the tests using the `make` command and the flag `WAVES=1` to generate a waveform-file.
4. Open the waveform file in a waveform viewer, e.g. GTKwave or Surfer

```
# Run the tests
(.venv) [<username>@<servername> test]$ make WAVES=1

# visualize the waveforms
(.venv) [<username>@<servername> test]$ gtkwave sim_build/adder.fst
```

0.1.2 Exercise B: Simple *cocotb* Test for Multiplexer Design

Objective: *Introduction to the cocotb tests*

Task: *Development of a cocotb test for simple RTL.*

In this test you will create two *cocotb* tests similar to the adder example for the multiplexer design (basic and random tests), using the test-setup found in

`lecture02/exercises/E02_intro_cocotb_exercisesE02_intro_cocotb_exercises/B_mux`.

Create the two *cocotb* tests in the test_mux.py file:

- `async def mux_basic_test(dut)`: Drive a single transaction through the DUT
- `async def mux_randomized_test(dut)`: Drive 10 random transactions thorugh the DUT

In the Makefile, add the test module to the `COCOTB_TEST_MODULES` make variable.

HINT: Look at the RTL for MUX. It has different ports than the adder!

Cocotb Triggers

Try using different methods form `cocotb.triggers` for allowing time to pass in simulation.

Example:

```
# generating clock signal, driving the clk of the DUT
Clock(signal=dut.clk,period=4,unit='ns').start()

# allowing time to pass
await Timer(2, 'ns')
await ClockCycles(dut.clk, 2)
await RisingEdge(dut.clk)
```

Add the different triggers to the end of the `mux_basic_test`.

0.1.3 Exercise C: Parallel *cocotb* Test

Objective: *Introduction to the cocotb tests.*

Task: *Development of a cocotb test for simple RTL.*

This exercise will introduce usage of `Combine` and `First`. Do the following exercise using the files in

`lecture02/exercises/E02_intro_cocotb_exercisesE02_intro_cocotb_exercises/C_parallel`

Create cocotb test for parallel design

Create a coroutine for each signal (A, B, C) that drives them at different intervals.

See example below.

```
async def drive_signal(dut):
    await RisingEdge(dut.clk)

    for _ in range(20):
        A = random.randint(0, 7)
        dut.A.value = A
        await ClockCycles(dut.clk, 3)

    dut.A.value = LogicArray('X'*4)
```

Create a test that starts all the coroutines using `cocotb.start_soon()`.

Use `Combine` to await all coroutines.

Similarly, create a test that starts all the coroutines and uses `First`.

- What are the differences between the two triggers? Read the docs at `https://docs.cocotb.org/en/v2.0.1/coroutines.html#`
- In what use cases could they be useful?